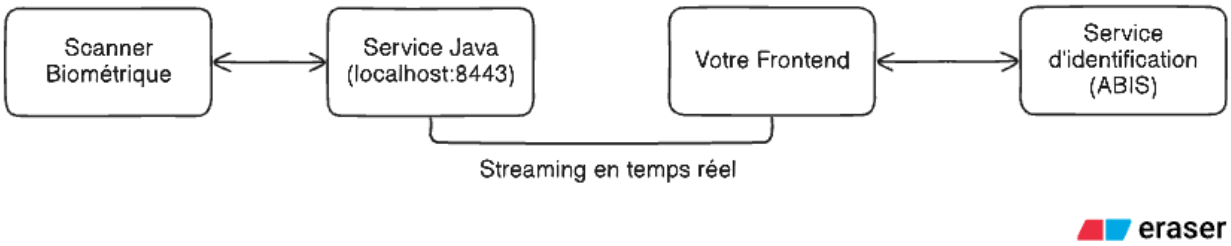
Guide d'Implémentation - Service de Capture Biométrique (Main Droite)

Vue d'ensemble

Ce document décrit l'architecture et les endpoints nécessaires à l'implémentation d'un service de reconnaissance des quatre doigts de la main droite (index, majeur, annulaire et auriculaire) à l'aide d'un scanner d'empreintes digitales.



Endpoints à utiliser

```
const BIOMETRIC_SERVICE = "https://localhost:8443";
const WEBSOCKET_URL = "wss://localhost:8443/ws/biometric-events";
const BACKEND_API = "https://10.30.30.22/api/Enrollement/check";
```

Étapes d'intégration

Etape 1 : Connexion WebSocket - WSS: /ws/biometric-events"

Objectif : Établir une connexion bidirectionnelle pour recevoir les événements du scanner.

```
// Créer la connexion WebSocket
const ws = new WebSocket("wss://localhost:8443/ws/biometric-events");

// Gérer l'ouverture de connexion
ws.onopen = () => {
  console.log("✅ Connecté au service biométrique");
  // Activer le bouton de capture dans votre UI
};

// Gérer les messages reçus
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  console.log("Message reçu:", data);

  // Traiter les différents types de messages (voir Étape 3)
};

// Gérer les erreurs
ws.onerror = (error) => {
  console.error("Erreur WebSocket:", error);
  // Afficher un message d'erreur à l'utilisateur
};

// Gérer la fermeture
ws.onclose = () => {
  console.log("🔌 Déconnecté du service biométrique");
  // Réessayer la connexion après 3 secondes
  setTimeout(() => {
```

```
    // Reconnecter...
  }, 3000);
};
```

Étape 2 : Déclencher la capture - GET : /capture-right-four

Objectif : Démarrer la capture des empreintes digitales.

```
async function startCapture() {
  try {
    // Appeler l'endpoint REST pour démarrer la capture
    const response = await fetch("https://localhost:8443/capture-right-four", {
      method: "GET"
    });

    if (!response.ok) {
      throw new Error(`Erreur HTTP: ${response.status}`);
    }

    console.log("Capture démarrée");
    // À partir de maintenant, vous allez recevoir des événements via WebSock
et
    // Le service Java va :
          // Allumer les LEDs du scanner
          // Attendre que l'utilisateur place ses doigts
          // Capturer les empreintes
          // Vous envoyer les images via WebSocket

  } catch (error) {
    console.error("Erreur lors du démarrage:", error);
    alert("Impossible de démarrer la capture. Vérifiez que le service est actif.");
```

```
  }
}
```

Étape 3 : Recevoir les images

Objectif : Récupérer les 5 images Base64 envoyées par le service.

Types de messages WebSocket

Type de message	Description	Exemple de données
IMAGE_PREVIEW	Aperçu en direct pendant le scan	{"imageData": "..."}
INSTRUCTION	Instructions pour l'utilisateur	"Placez vos doigts sur le scanner"
FINGERS_BASE64_IMG	Image d'un doigt (x5)	""
CAPTURE_ERROR	Erreur durant la capture	"Qualité insuffisante"
CAPTURE_DONE	Capture terminée	"Capture réussie"

```
// Tableau pour stocker les 5 images
let capturedImages = [];

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  const { type, message } = data;

  switch (type) {
    case "IMAGE_PREVIEW":
      // Afficher l'aperçu en temps réel
      const preview = JSON.parse(message);
      displayPreview(preview.imageData);
```

```
      break;

    case "INSTRUCTION":
      // Afficher les instructions à l'utilisateur
      showInstruction(message); // Ex: "Placez vos 4 doigts..."
      break;

    case "FINGERS_BASE64_IMG":
        // Stocker l'image reçue
        // NB : Il faut savoir que l'envoi des images se fera de manière séquentiell
e.
          // C'est-à-dire qu'elles seront transmises une par une, en commençant
par l'image
          // de la main complète, suivie des images des doigts (une image par do
igt),
          // de gauche à droite.
      capturedImages.push(message);
      console.log(`Image ${capturedImages.length}/5 reçue`);

      // Afficher la première image (main complète) comme aperçu
      if (capturedImages.length === 1) {
        displayCapturedImage(message);
      }
      break;

    case "CAPTURE_DONE":
      // Toutes les images ont été reçues
      console.log("Capture terminée, 5 images reçues");
      enableSendButton(); // Activer le bouton d'envoi
      break;

    case "CAPTURE_ERROR":
      // Gérer les erreurs
      console.error("Erreur:", message);
      showError(message);
      break;
```

```
  }
};
```

Ordre de réception des images

```
Image 1 : Main droite complète (4 doigts ensemble)
Image 2 : Index droit (doigt isolé)
Image 3 : Majeur droit (doigt isolé)
Image 4 : Annulaire droit (doigt isolé)
Image 5 : Auriculaire droit (doigt isolé)
```

Recommandation : Affichez la première image (main complète) à l'utilisateur comme confirmation visuelle.

Étape 4 : Convertir et envoyer les images

Objectif : Transformer les images Base64 en fichiers et les envoyer au backend.

A. Fonctions de conversion

Dans ce contexte, la fonction `base64ToBlob` a pour rôle de convertir les images capturées au format Base64 en fichiers binaires exploitables. Cette conversion est indispensable, car une chaîne Base64 ne peut pas être envoyée directement au backend dans ce cas. `base64ToBlob` permet ainsi de transformer les images issues du scanner en un format binaire conforme aux attentes du backend pour le traitement de la reconnaissance.

```
/**
 * Convertit une chaîne Base64 en Blob
 */
```

```
function base64ToBlob(base64String, contentType = "image/png") {
  // Retirer le préfixe "" si présent
  const base64 = base64String.includes(",")
    ? base64String.split(",")[1]
    : base64String;

  // Décoder le Base64
  const byteCharacters = atob(base64);
  const byteNumbers = new Array(byteCharacters.length);

  for (let i = 0; i < byteCharacters.length; i++) {
    byteNumbers[i] = byteCharacters.charCodeAt(i);
  }

  const byteArray = new Uint8Array(byteNumbers);
  return new Blob([byteArray], { type: contentType });
}
```

B. Préparation du FormData

La fonction prepareFormData a pour rôle de transformer et organiser les images capturées afin de produire un FormData conforme au attentes du backend, prêt à être envoyé pour le traitement de reconnaissance biométrique.

```
/**
 * Prépare les images pour l'envoi au backend
 */
function prepareFormData(capturedImages) {
  // IMPORTANT : Noms exacts attendus par le backend
  const fingerNames = [
    "RightHand",      // Image 1 : Main complète
    "RightIndex",     // Image 2 : Index
    "RightMiddle",    // Image 3 : Majeur
    "RightRing",      // Image 4 : Annulaire
```

```
    "RightLittle"     // Image 5 : Auriculaire
  ];

  const formData = new FormData();

  // Convertir chaque image et l'ajouter au FormData
  capturedImages.forEach((imageBase64, index) => {
    const blob = base64ToBlob(imageBase64);
    const filename = `${fingerNames[index]}.png`;

    // CRITIQUE : Le paramètre DOIT s'appeler "request"
    formData.append("request", blob, filename);
  });

  return formData;
}
```

C. Envoi au backend - POST : /api/Enrollement/check

```
/**
 * Envoie les empreintes au backend pour identification
 */
async function sendFingerprints(capturedImages) {
  try {
    // Vérifier qu'on a bien les 5 images
    if (capturedImages.length !== 5) {
      throw new Error(`Images incomplètes: ${capturedImages.length}/5`);
    }

    // Préparer le FormData
    const formData = prepareFormData(capturedImages);
```

```
    console.log("Envoi des empreintes au backend...");

    // Envoyer la requête
    const response = await fetch("https://10.30.30.22/api/Enrollement/check", {
      method: "POST",
      body: formData
    });

    // Traiter la réponse
    if (response.status === 200) {
      // Personne identifiée
      const person = await response.json();
      console.log("Personne trouvée:", person);
      displayPersonInfo(person);

    } else if (response.status === 404) {
      const text = await res.text()
      console.log(text)
    } else {
      //  Autre erreur
      throw new Error(`Erreur HTTP: ${response.status}`);
    }

  } catch (error) {
    console.error("❌ Erreur lors de l'envoi:", error);
    showError("Impossible de vérifier les empreintes. Veuillez réessayer.");
  }
}
```

Réponse 200 (Personne trouvée)

```
{
  "idPersonne": "12345",
```

```
  "numeroAssure": "A123456",
  "numeroBeneficiaire": "B789012",
  "nom": "DUPONT",
  "prenom": "Jean",
  "sexe": "M",
  "dateNaiss": "1985-05-15T00:00:00",
  "hasRight": true,
  "ftpFace": "/images/faces/12345.jpg",
  "dateMariage": null,
  "beneficiaire": [
    {
      "numeroBeneficiaire": "B789013",
      "nom": "DUPONT",
      "prenom": "Marie",
      "sexe": "F",
      "dateNaiss": "1987-08-20T00:00:00",
      "lienParente": "Conjoint(e)",
      "hasRight": true,
      "ftpFace": "/images/faces/67890.jpg"
    }
  ]
}
```